

# task2 report.pdf

*by* Rongan CHEN

---

**Submission date:** 15-Apr-2026 01:17PM (UTC+0800)

**Submission ID:** 2932531758

**File name:** task2\_report.pdf (1.83M)

**Word count:** 1584

**Character count:** 7266

**Requirement on academic honesty (COMP2090SEF):**

1. Each group must make the following declaration on the cover page of the report

Hong Kong Metropolitan University

COMP2090SEF  
Data Structures, Algorithms And Problem Solving

**Course Project Report**  
Group 69

Submission date: April 17,2026

We declare that:

- (i) all members of the group have read and checked that all parts of the project (including proposal, code programs, and reports), irrespective of whether they are contributed by individual member or all members as a group, here submitted is original except for source material explicitly acknowledged;
- (ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration;
- (iii) we are aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;
- (iv) we confirm that we have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and
- (v) we are aware that all members of the group should be held responsible and liable to disciplinary actions, irrespective of whether he/she has signed the declaration and whether he/she has contributed, directly or indirectly, to the problematic contents.

We confirm that we have read through and understood the project requirements. We understand that failure to comply with the project requirements will result in score deduction.

| NAME        | SID      |
|-------------|----------|
| Shi Yifan   | 14153433 |
| Han Xiaoyu  | 13798919 |
| Chen Rongan | 14092908 |

2. The group must acknowledge ALL external resources or code from others referenced. We will only mark the parts that are original, not the parts from others.
  3. All members are liable to the plagiarism incidence, even if a free-rider.
-

This is the group report for Task 2 of COMP2090SEF.

## I. Definition of Heap

1. A heap must be a complete binary tree. So, what is a complete binary tree?

A complete binary tree has the following properties: (1) A complete binary tree only allows the last row of elements to be incomplete. (2) The last row of a complete binary tree must be sorted from left to right. (3) The middle of the last row of a complete binary tree cannot have any gaps. Let's look at some specific examples: (Figure1) All the three trees above conform to the definition of a complete binary tree, so they are all complete binary trees. (Figure2) The above trees do not conform to the definition of a complete binary tree, so they are not complete binary trees.

As for why a heap must be a complete binary tree, this will be explained in the following part of the report.

2. Heap Order: Based on heap order, heaps can be classified into two types: max heap and min heap. In a max heap, each parent node element must be greater than its child node elements. In a min heap, each parent node element must be less than its child node elements.

ADD: Heap ADT Interface Definition:

A heap is an abstract data type that satisfies the structure of a complete binary tree and heap properties. It is divided into maximum heap and minimum heap. The standard interface definition for it is as follows: Create an empty heap to initialize an empty heap without any elements; Insert an element, add a new value to the heap and automatically maintain the heap structure; Get the top element of the heap, which returns the maximum value (for maximum heap) or the minimum value (for minimum heap) without deleting the element; Delete the top element of the heap, remove the top value and re-adjust the heap structure; Check if the heap is empty, return a boolean value indicating the empty/full state of the heap; Get the heap size, return the total number of elements in the heap; Build a heap in batches, quickly build a complete heap structure by inputting an array; Clear the heap, delete all elements in the heap and restore it to an empty heap.

## II. Storage of the Heap

Next, let's consider how this data structure is actually stored.

(Figure3) First, we assign numbers to the nodes in the order of level-order traversal (from top to bottom, from left to right).

These numbers are then mapped to the indices in an array, and the elements of the tree are stored in the corresponding indices. Since the heap is a complete binary tree, each index is one-to-one corresponding to the position of the tree. Thus, a heap can be described by a one-dimensional array: (Figure4)

We can observe the pattern between the node indices: if the node index is  $i$ , then the index of its left child node is  $2i + 1$ , and the index of its right child node is  $2i + 2$ . This pattern is frequently used in the algorithm.

## III. Basic Operations of Heap:

A heap has two basic operations, namely up-filtering and down-filtering. By using these two operations, almost all functions of a heap can be achieved.

Example: Consider a tree where only the root element does not satisfy the heap property (Figure 5), while the elements below all do. How can we adjust the tree to become a heap?

Take the maximum heap as an example. We compare the element that violates the heap property with its largest child node. If it is smaller than the largest child node, we swap them and continue the comparison and swapping until the element is larger than its child nodes, or until it reaches the bottom. At this point, the tree has successfully been transformed into a maximum heap. The operation of moving the root node downward is called down-filtering. It is easy to see that the complexity is  $O(\log N)$ .

When only the last element of the tree violates the heap property (Figure 6), treat it and its parent element in the same way and compare them. If the element is greater than the parent node, swap them. Continue this process until it cannot be moved up any further. This operation is called "upward filtering" and is mainly used for inserting new elements into the heap. Its complexity is  $O(\log N)$ .

#### IV. Building the heap:

There are two methods for building a heap: the top-down heap-building method and the bottom-up heap-building method.

They correspond to the top-down filtering and the bottom-up filtering respectively.

The top-down heap-building method involves inserting elements one by one into the heap and then performing top-down filtering. The time complexity is  $O(N \log N)$ .

The bottom-up heap-building method involves first adjusting the lower elements to form a heap, and then performing bottom-up filtering on each parent node until the root node.

This heap-building method has a complexity of  $O(N)$ .

#### V. Specific Applications of Heap:

1. Priority Queue - Remove the smallest element. This application can be implemented using a min-heap. Remove the root node (the smallest element) of the min-heap, and then adjust the remaining elements into a heap (place the last element at the root node and perform the down-filtering). The complexity of this operation is  $O(\log N)$ .
2. Insertion Operation - Essentially, it is the up-filtering process, with a complexity of  $O(\log N)$ .

#### VI. Heap Sort:

Theoretically, all elements of the priority queue can be popped one by one.

However, considering the space complexity, the sorted result will be stored in the vacant cells of the heap.

Specific steps:

1. Transform the unordered array into a binary heap.
2. Continuously remove the top element of the heap, move it to the end of the set, and adjust the heap to generate a new top element.

4 When we remove the top element of the maximum heap (not completely delete it, but replace it at the very end), through self-adjustment, the second largest element will be swapped up and become the new top element of the maximum heap.

Due to this characteristic of the binary heap, every time we remove the old top element and adjust the new top element after adjustment, it will always be the node with the second largest size after the old top element.

As long as we repeatedly deleted the top of the heap and repeatedly adjust the binary heap, the resulting set will become an ordered set.

Complexity:

8 Heap sorting is an unstable sorting method. Its space complexity is  $O(1)$ , and the average time complexity is  $O(N\log N)$  (Figure 7) because in each round of sorting, we need to perform a downward adjustment, and it must be done from the root node downwards. In the worst case, it remains stable.  $O(N\log N)$

#### Heap Sort Step-by-Step Demonstration

Take the array [4, 6, 8, 5, 9] as an example, using the maximum heap to achieve ascending order sorting. The complete steps are as follows:

1. Initial sorted array: [4, 6, 8, 5, 9]
- 5 2. Step 1: You should build the maximum heap. Start from the last non-leaf node and adjust downward, converting the array into a maximum heap structure. After the adjustment, the maximum heap is: [9, 6, 8, 5, 4]
3. Step 2: Swap the top of the heap with the last element, fixing the max value at the end. Swap the elements at index 0 and index 4, the array becomes: [4, 6, 8, 5, 9], at this point, the last 9 has been sorted, the remaining sorted interval is the first 4 elements [4, 6, 8, 5]
4. Reheapify the first 4 elements to restore the max heap structure, resulting in: [8, 6, 4, 5, 9]
- 1 5. Swap the top of the heap with the current element at the end of the sorted interval, swap the elements at index 0 and index 3, the array becomes: [5, 6, 4, 8, 9], at this point, 8 has been sorted, the remaining sorted interval is the first 3 elements [5, 6, 4]
6. Reheapify the first 3 elements to restore the max heap structure, resulting in: [6, 5, 4, 8, 9]
7. Swapping the top of the heap with the current element at the end of the sorted interval, swap the elements at index 0 and index 2, the array becomes: [4, 5, 6, 8, 9], at this point, 6 has been sorted, the remaining sorted interval is the first 2 elements [4, 5]
8. Reheapify the first 2 elements to restore the maximum heap structure, resulting in: [5, 4, 6, 8, 9]
- 1 9. Swapping the top of the heap with the current element at the end of the sorted interval, swap the elements at index 0 and index 1, the array becomes: [4, 5, 6, 8, 9]
10. All elements have been sorted, the final ascending order array is: [4, 5, 6, 8, 9]

Regarding the Python code implementation of heap build heap sort, I will add it to the code in the Github repository.

GitHub Link:

[https://github.com/3031445204zelsy17-lang/COMP2090SEF\\_group\\_project\\_GRP\\_69](https://github.com/3031445204zelsy17-lang/COMP2090SEF_group_project_GRP_69)

Appendix:  
Figure1:

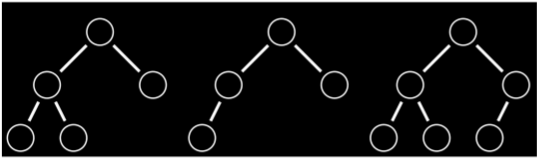


Figure2:

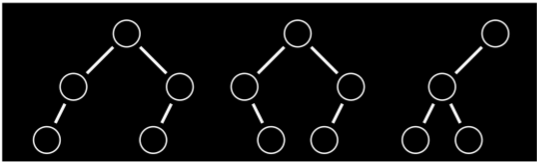


Figure3:

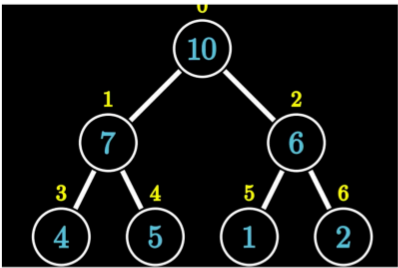


Figure4:



Figure5:

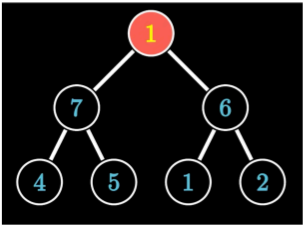


Figure6:

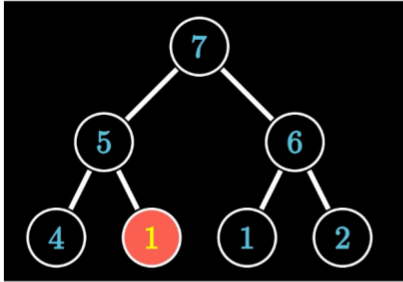


Figure7:

```
def heap_sort(arr):  
    """  
    Heap Sort (Ascending Order): Implemented based on max-heap  
    Overall Time Complexity:  $O(n \log n)$  (Heap building  $O(n)$  +  $n$  heapify operations  $O(\log n)$ )  
    Space Complexity:  $O(1)$  (In-place sorting, no extra array space needed)  
    """  
    n = len(arr)  
    # Step 1: Build max-heap from input array -  $O(n)$   
    build_max_heap(arr)  
  
    # Step 2: Extract elements from heap one by one  
    for i in range(n - 1, 0, -1):  
        # Swap root (max element) with the last element of current heap  
        arr[i], arr[0] = arr[0], arr[i]  
        # Heapify the reduced heap (heap size = i) -  $O(\log n)$  per operation  
        heapify(arr, i, 0)  
  
    return arr
```



Thanks:

Figure 1-6 Source :

【【从堆的定义到优先队列、堆排序】10分钟看懂必考的数据结构——堆】[https://www.bilibili.com/video/BV1AF411G7cA?vd\\_source=e77e19a1fe90e43d92721213c1134ce3](https://www.bilibili.com/video/BV1AF411G7cA?vd_source=e77e19a1fe90e43d92721213c1134ce3)

ORIGINALITY REPORT

16%

SIMILARITY INDEX

9%

INTERNET SOURCES

6%

PUBLICATIONS

9%

STUDENT PAPERS

PRIMARY SOURCES

|    |                                                                                                                |    |
|----|----------------------------------------------------------------------------------------------------------------|----|
| 1  | Submitted to Xiamen University<br>Student Paper                                                                | 3% |
| 2  | Dinesh P. Mehta, Sartaj Sahni. " Handbook of Data Structures and Applications", CRC Press, 2018<br>Publication | 2% |
| 3  | Submitted to Manchester Metropolitan University<br>Student Paper                                               | 2% |
| 4  | Submitted to Coventry University<br>Student Paper                                                              | 2% |
| 5  | fastercapital.com<br>Internet Source                                                                           | 1% |
| 6  | link.springer.com<br>Internet Source                                                                           | 1% |
| 7  | sitams.ac.in<br>Internet Source                                                                                | 1% |
| 8  | www.bartleby.com<br>Internet Source                                                                            | 1% |
| 9  | www.darshan.ac.in<br>Internet Source                                                                           | 1% |
| 10 | www.fatalerrors.org<br>Internet Source                                                                         | 1% |
| 11 | github.com<br>Internet Source                                                                                  | 1% |

12

Internet Source

<1 %

13

[www.coursehero.com](http://www.coursehero.com)

Internet Source

<1 %

Exclude quotes Off

Exclude matches Off

Exclude bibliography Off